Ecolectro Server

For IT

Samantha Sikora & Issak Nathan

Contents:

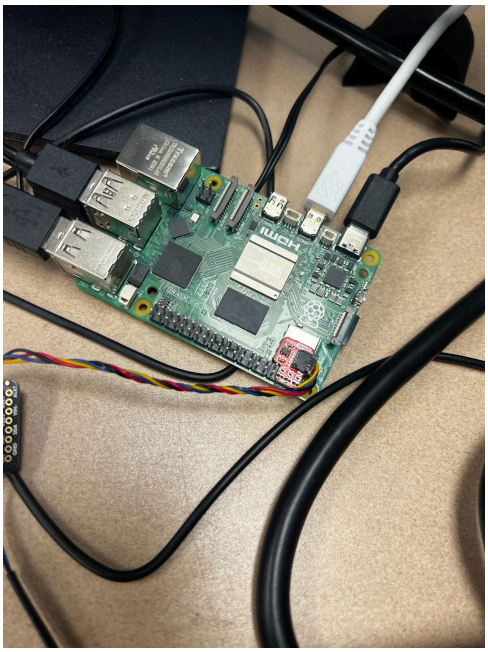
1. Purpose
2. Hardware Setup
3. Applications & Services
4. Access Rules
5. Testing and monitoring
6. Code

Purpose:

This server functions as a central observability and backup node similar to a SCADA system, designed to monitor applications, store logs, and back up critical data. It operates within a self-contained environment, suitable for both on premise and remote use for the administrator. Clients will be able to view data that is controlled by the administrator and safe for the public to view while also keeping track, and observing abnormalities, with safety features from a firewall.

The server utilizes several interconnected tools, forming a lightweight yet powerful stack ideal for small businesses, like Ecolectro. It will be a good starting point for future endeavors in protecting, maintaining and performing advanced data systems.

Hardware & Networking:



Devices in use:

-Mini PC dell optiplex 3080 with Debian

~PCIe 4.0 NVMe M.2 SSD (2TB)

-Netgear GS105 Gigabit Switch

-Access point to internet

-2 minimum ethernet cords

~PC to gigabit switch

~Switch to access point

Extra devices:

-Mini PC dell optiplex 3080 with Promox

-Raspberry pi 5

-ethernet cord

~Pi to gigabit switch

-SATA 2.5-inch SSD BX500 (2TB)

Recommend upgrades:

-larger external/internal TB storage for the mini PC

-larger GPU

-16G RAM

Applications & Services:

- Metrics Monitoring

~Helps with: Tracks real-time system and service performance (CPU, memory, disk, network, etc.).

~Future use is collecting data

~How:

- Prometheus scrapes metrics from system exporters or services (like node_exporter or prometheus client).
- VictoriaMetrics stores this time-series data in a high-performance, low-resource database.
- Grafana visualizes the data in dashboards, allowing alerts and analysis.

- Log Aggregation

~Helps with: Collects and centralizes logs from services, system files, or custom applications.

~How:

- Fluent Bit is the log collector. It tails files (e.g. custom logs) and forwards logs to Loki.
- Loki stores logs with timestamp and labels (like service name or severity).
- Logs are queryable and visualizable via Grafana.

- Data Backup & Sync

~Helps with: Ensures that valuable data (metrics, configs, logs, databases) is not lost.

~How:

- A scheduled script uses Rclone to upload or sync specified directories to Google Drive.
- Backup logs are retained and monitored.
- Offline or low-bandwidth options supported via local storage.

-What needs to be improved for data backup:

~Make the data back up automatically, at the current moment you have to manually back the data up in order for it to save. Here are different ideas you could use:

~Borg using Cron

~TrueNas

~Python script for backups

~Use the 3-2-1-1-0 Rule: Have at least three backups of your data on two different media, with one off-site and one that is offline, air-gapped, or immutable for zero errors after automated backup testing and recoverability verification with Veeam Data Platform.

- Remote & Secure Access

~Help with: Allows access to the server and dashboards from outside the local network.

~How:

- Tailscale creates a private, encrypted mesh VPN.

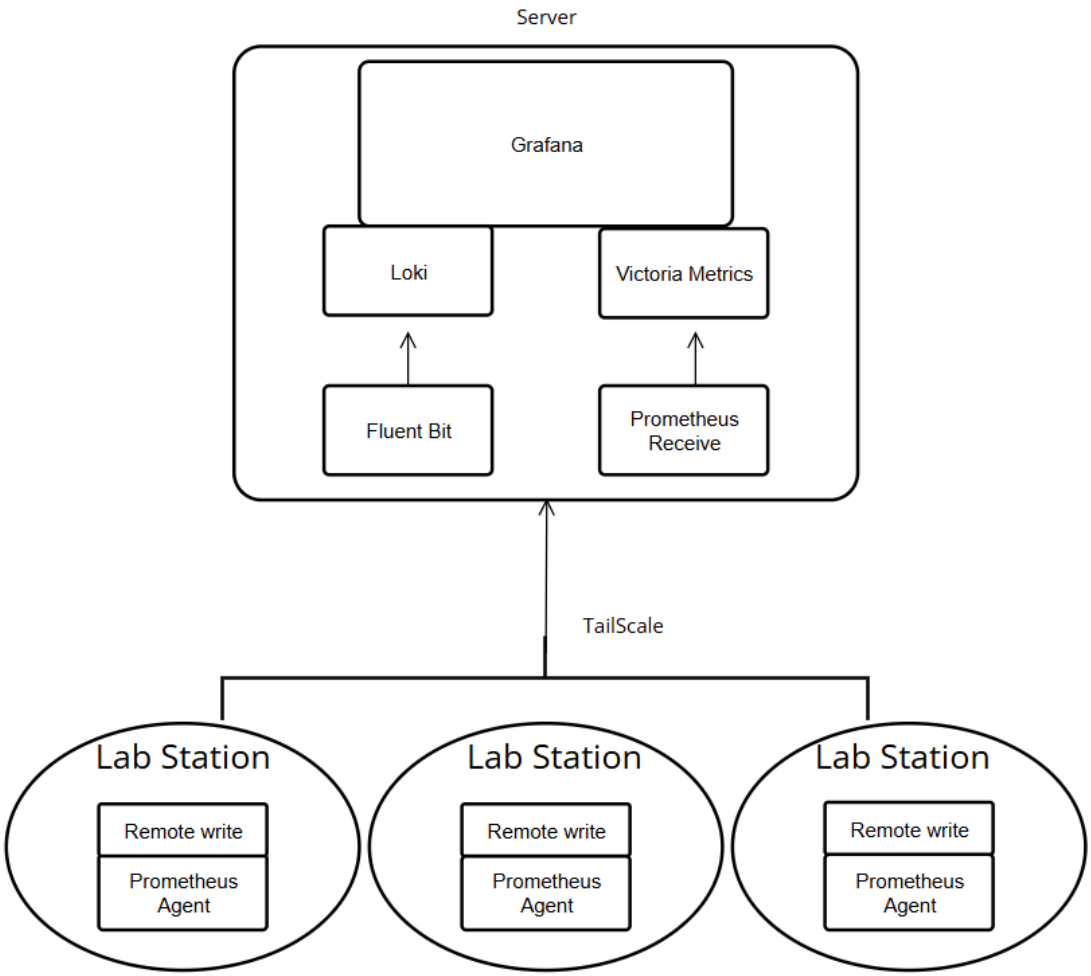
  This means the server can be accessed securely without needing port forwarding or public exposure.

- You can SSH in or view Grafana dashboards from anywhere with Tailscale installed.

-What needs to be improved:

~put up a firewall for more security

~use ethernet for just admin access, separate from main network



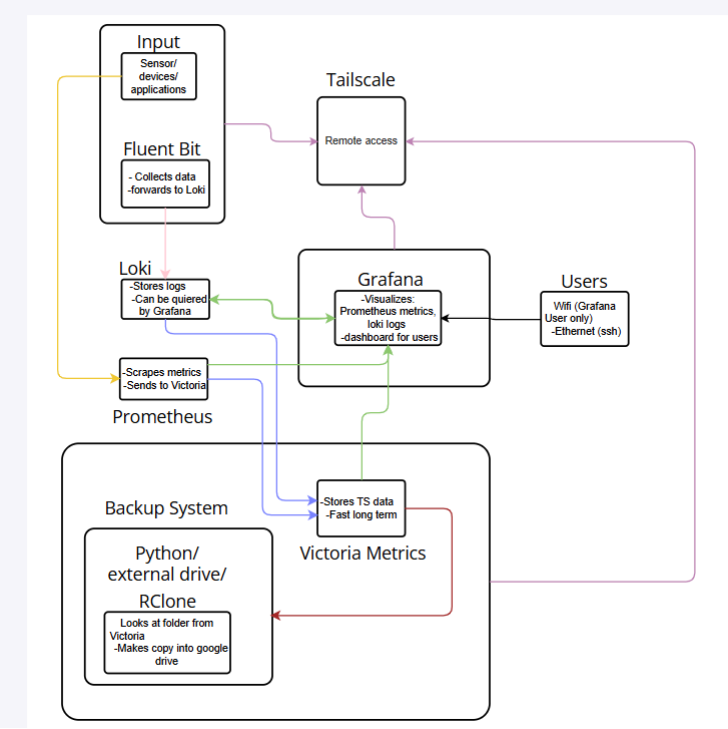
Access Rules:

-**Grafana** – Accessible over Wi-Fi

-**Prometheus & Loki** – Accessible only via Tailscale for security

-**Backups** – Run daily and upload with Rclone

-Need to be done: - **SSH** – Only over Ethernet (firewalled from Wi-Fi)



Testing & monitoring:

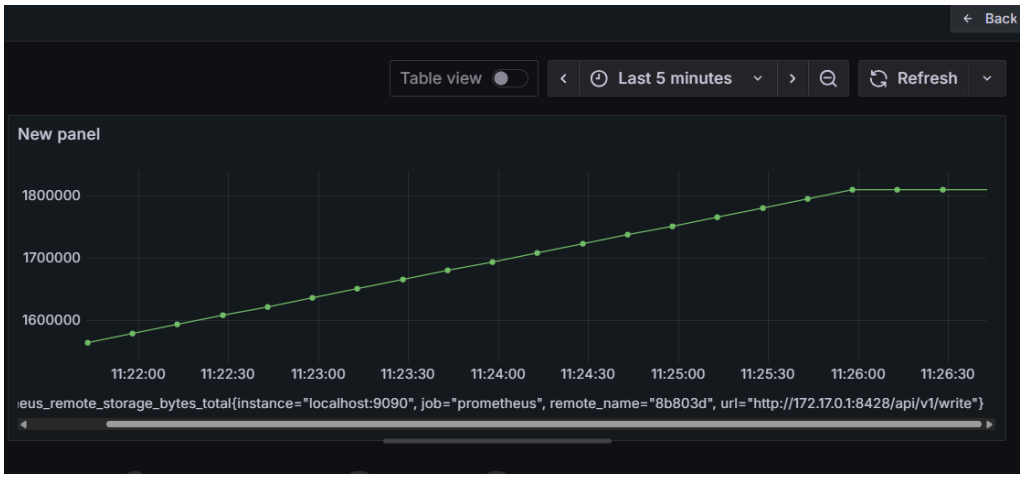
-Subject to change IPs:

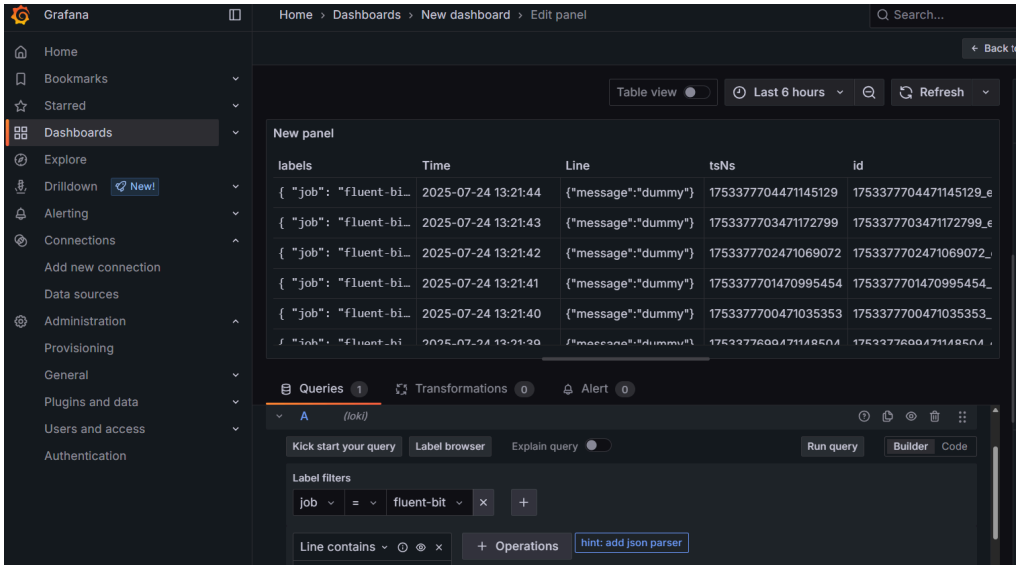
-Grafana: http://192.168.252.33:3000

~user: admin

~password: ecolectro or admin

~By using queries from loki, it will connect to a fluent bit and print the following logs from specific programs or files that are listed.





-VictoriaMetrics: http://192.168.252.33:8428

-Prometheus: http://192.168.252.33:9090

-Loki: http://loki:3100

-Pi webserver: 192.168.252.146:3000

-Proxmox (only on the extra mini PC): 192.168.1.250:8006

~user: root

~password: ecolectro2025

Code:

-fluent-bit.conf

```
#[Service] section defines the global parameters of the fluent bit service
[SERVICE]
    Flush        1                  # Interval (in seconds) to flush logs to output
    Daemon       Off                # Run as a foreground process (set to On to run as a daemon/background process)
    Log_Level    debug              # Set the logging verbosity (trace,debug info, error, warn)
    Parsers_File parsers.conf       # Load parser definitions from the given file

# [Input] section defines where Fluent But reads logs from
[INPUT]
    Name              tail          # Use the 'tail' plugin to read log files line by line
    Tag               docker.*      # Tag format for incoming logs (used for routing/matching)
    Path              /var/lib/docker/containers/*/*.log        #Path to Docker container log files
    Parser            docker        # Use the 'docker' parser (defined in parsers.conf)
    Mem_Buf_Limit     5MB           # Memory buffer limit per file
    Skip_Long_Lines   On            #Skip log lines longer than the buffer limit instead of crashing
    Refresh_Interval  1             # Interval (in seconds) to scan for new files

# [Output] section defines where Fluent Bit sends logs
[OUTPUT]
    Name         loki               # Use the 'loki' plugin to forward logs to Grafana Loki
    Match        *                  # Match all tags (i.e., send all logs)
    Host         loki               # Hostname or IP of the Loki service (e.g., in Docker network)
    Port         3100               # Loki's HTTP ingestion port (default:3100)
    Labels       job=fluent-bit     # Static label to apply to all logs (can be used in Grafana queries)

# Additional [Output] to also print logs to standard output for debugging
[OUTPUT]
    Name stdout                     # Use the 'stdout' plugin to print logs to the console
    Match *                         # Match all tags (i.e., print all logs)
```

-prometheus.yml

```yaml
# Global configuration settings for Prometheus
global:
  scrape_interval: 15s        # Default interval to scape targets (every 15 seconds)

# Congifuration for sending collected metrics to a remote storage (VictoriaMetrics in this case)
remote_write:
  - url: http://172.17.0.1:8428/api/v1/write  # URL of the remote storage endpoint (VictoriaMetrics write API)

# List of scrape configurations for Prometheus
scrape_configs:

# First job: Prometheus monitoring itself
  - job_name: "prometheus"    # Name of the scrape job

    static_configs:
      - targets: ["localhost:9090"]   # Prometheus server's own web interface (default port 9090)

# Second job: Collecting data from FlowMass Sensor
  - job_name: "FlowMass Sensor Data"

    # Override global settings
    scrape_interval: 1s

    static_configs:
      - targets: ["100.94.48.102:8000"]   #IP and port of pi
```

-docker-compose.yml

```yaml
#Define the networks used for container communication
networks:
  grafana:
    driver: bridge    # Uses the default brdige network for isolation

#Define shared volumes used by services for data persistence
volumes:
  vmdata: {}          #Placeholder(currently unused)
  grafanadata: {}     # Stores persistent Grafana data

▷Run All Services
services:
  # VictoriaMetrics instance, a single process responsible for
  # storing metrics and serve read requests.
  ▷Run Service
  victoriametrics:
    image: victoriametrics/victoria-metrics
    networks:
      - grafana
    ports:
      - 8428:8428       #Expose the web and ingestion API
    volumes:
      - ./VicMet_Data:/storage    #Mount local directory to container for data storage
    command:
      - "--storageDataPath=/storage"    #Location for storing metrics data
      - "--httpListenAddr=:8428"        #Listening port for HTTP server
      - "-search.disableCache"          #Disable query cahcing
      - "-retentionPeriod=100"          #Retain metrics for 100 days
    restart: always
```

```yaml
  #Prometheus collects metrics and can forward them to VictoriaMetrics
  ▷Run Service
  prometheus:
    image: prom/prometheus:main
    volumes:
      - ./prometheus/config/prometheus.yml:/etc/prometheus/prometheus.yml     #Prometheus config
      - ./prometheus/data:/etc/prometheus/data                                # Local path to store time series data
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"        #Load config from mapped file
      - "--storage.tsdb.path=/etc/prometheus/data"            #Where Prometheus stores data
    ports:
      - 9090:9090 #Prometheus web UI
    networks:
      - grafana
    # restart: always

  #Loki is a log aggregation system that works well with Grafana
  ▷Run Service
  loki:
    image: grafana/loki:latest
    ports:
      - 3100:3100   #Loki HTTP Api port
    # volumes:
    #   - ./loki/local-config.yaml:/etc/loki/local-config.yaml
    # command: "-config.file=/etc/loki/local-config.yaml"
    networks:
      - grafana
```

```yaml
  #Granfana provides dashboards and visualiaztion for metrics/logs
  ▷Run Service
  grafana:
    image: grafana/grafana:main
    depends_on:
      - "victoriametrics"   #wait for VictoriaNetrics to be ready before starting Grafana
    ports:
      - 3000:3000   #Grafana web interface
    networks:
      - grafana
    environment:
      - "GF_DEFAULT_APP_MODE=development"   #Development mode (e.g., for more verbose logging)
      - "GF_LOG_LEVEL=debug"                #Set logging to debug level
    volumes:
      - grafanadata:/var/lib/grafana        #Persist Grafana state (dashboards, users)
      - ./grafana/provisioning/datasources:/etc/grafana/provisioning/datasources    #Preconfigured data sources
      - ./grafana/grafana.ini:/etc/grafana/grafana.ini
    restart: always

  #Rclone syncs VictoriaMetrics data directory to a remote drive periodically (e.g., Google Drive)
  ▷Run Service
  rclone:
    image: rclone/rclone:latest
    networks:
      - grafana
```

```yaml
    volumes:
      - ./rclone:/config/rclone        # rclone config (rclone.conf file)
      - ./VicMet_Data:/storage         #Source directory to backup
    entrypoint: ["/bin/sh", "-c"]
    command: >
      "while true; do
       rclone sync -L /storage/snapshots Drivtest:backup --config /config/rclone/rclone.conf && sleep 3600;
       done"
    #Infinite loop: sysnc data every hour from local to remote storage
    restart: always

#Fluent bit collects and forwards logs (e.g., to Loki or to file)
▷Run Service
fluentbit:
  image: fluent/fluent-bit
  depends_on:
    - loki      #Fluent Bit waits for Loki to be available
  networks:
    - grafana
  volumes:
    - ./fluentbit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf   #Fluent bit config file
    - ./fluentbit/outpot.log:/var/log/output.log                    #Log file source
  command: ["--config=fluent-bit/etc/fluent-bit.conf"]              #Specify config file path
  restart: always
```

```yaml
#Flog is a fake log generator used to test Fluent Bit + Loki setup
▷Run Service
flog:
  image: mingrammer/flog
  volumes:
    - ./fluentbit/outpot.log:/var/log/output.log    #Write logs to the file Fluent bit is reading from
  command: ["flog -t log -o /var/log/output.log -wc -n 100"]
```